

StudyMate

Projektdokumentation

Karteikarten-basierte Web-Lernplattform

DHBW Mannheim · TINF24CS2 · Modul Software Engineering (WS 2025/26)

Gruppe: Selina Ickstadt · Devi Müller · Michelle Schneider · Jennifer Hirt

Dozent: Jürgen Schultheis

Abgabe: 26.06.2026

Version 0.3.2 · git main (PR #63)

Inhaltsverzeichnis

1 Projektüberblick.....	3
2 Projektmanagement & Teamorganisation.....	4
3 Anforderungen & User Stories.....	7
4 Architektur & Technologie-Stack.....	19
5 Datenbankmodell.....	22
6 Backend — Komponenten & API-Referenz.....	26
7 Frontend — Komponenten & Funktionen.....	29
8 Authentifizierung & Sicherheit.....	32
9 Tests & Qualitätssicherung.....	33
10 Feature-Dokumentation (chronologisch).....	36
11 UI/UX — Design & Animationen.....	39
12 CI/CD & Deployment.....	40
13 Lokale Entwicklungsumgebung.....	41
14 Troubleshooting & FAQ.....	42
15 Versionshistorie (ChangeLog).....	43
16 Offene Aufgaben & Ausblick.....	45
Anhang — Referenzen.....	46

1 Projektüberblick

StudyMate ist eine vollständige, modern gestaltete Web-Lernplattform mit Fokus auf karteikarten-basiertes Lernen. Das Projekt entstand im Rahmen des Moduls Software Engineering (WS 2025/26) als kollaboratives Teamprojekt und vereint agile Entwicklung, einen sauberen Git-Workflow und die Integration externer Cloud-Dienste.

Kernziele

- Leichtgewichtige, erweiterbare Plattform für karteikarten-basiertes Lernen bereitstellen.
- Moderne Software-Engineering-Methoden praxisnah anwenden (agile Entwicklung, Git-Workflow, CI/CD).
- Externe Dienste sinnvoll integrieren: Supabase (Auth & Datenbank), Groq KI (Quiz), Resend (E-Mail).

Implementierte Hauptfunktionen

Funktion	Beschreibung	Implementiert am
Authentifizierung	Login, Registrierung, Gastmodus, Passwort-Reset (OTP)	2026-05-19
Lernset-Verwaltung	Erstellen, bearbeiten, löschen, Sichtbarkeit umschalten	2026-05-19 / 05-26
Karteikarten-Verwaltung	Anlegen, bearbeiten, löschen, JSON-Import, Reihenfolge	2026-05-19 / 06-02
Lernmodus	Flip-Card-Ansicht mit Fortschrittsanzeige und Streak	2026-05-19
Standard-Quiz	Multiple-Choice auf Basis vorhandener Karten	2026-05-19
KI-Quiz	Quiz-Generierung via Groq API (Llama) mit Erklärungen	2026-05-26 / 06-02
Fork-Funktion	Öffentliche und eigene Sets kopieren (inkl. Priv-zu-Priv)	2026-05-26 / 06-02
Favoritensystem	Sets als Favorit markieren (auch als Gast), Migration bei Login	2026-05-19 / 06-02
Streak-Tracking	Tägliche Lernserie, persistiert im Browser	2026-05-26
Soziale Funktionen	Freunde, Freundschaftsanfragen, öffentliche Profile	2026-06-02
Geführte Tour	Interaktives Onboarding für Erstnutzer	2026-05-26 / 06-02
Profilbild in NavBar/Sidebar	Hochgeladenes Profilbild erscheint im Avatar	2026-06-01
Light/Dark-Mode	Umschaltbares Farbschema, gespeichert per localStorage	2026-05-26

2 Projektmanagement & Teamorganisation

StudyMate wurde von einem vierköpfigen Studierendenteam nach agilen Prinzipien entwickelt. Als organisatorischer Rahmen diente das aus der Vorlesung bekannte Scrum-Framework in Kombination mit einem Kanban-Board zur Visualisierung des Arbeitsflusses. Das Team arbeitete selbstorganisiert und integrierte seine Ergebnisse über einen Git-Workflow mit Code-Reviews.

Team-Aufbau

Da es sich um ein gleichberechtigtes Studierendenteam handelt, wurden die klassischen Scrum-Rollen gemeinsam bzw. rollierend wahrgenommen. Der Dozent fungierte als Stakeholder und nahm die Ergebnisse ab.

Teammitglied	Arbeitsbranch
Selina Ickstadt	Selina
Devi Müller	devi
Michelle Schneider	Michelle
Jennifer Hirt	Jenny

Rolle	Umsetzung im Projekt
Product Owner	Backlog pflegen & User Stories priorisieren (gemeinsam)
Scrum Master	Prozess & Board-Pflege sichern, Hindernisse beseitigen (rollierend)
Entwicklerteam	Alle 4 Mitglieder; autonome, selbstorganisierte Umsetzung
Stakeholder	Dozent (Auftraggeber, Feedback & Abnahme)

Agiler Prozess: Meetings & Artefakte

Das Vorgehen folgte dem Scrum-Prozess aus wiederkehrenden Iterationen (Sprints), die jeweils durch feste Meetings strukturiert waren.

Meeting	Zweck
Planning	Aufgaben auswählen, schätzen, Sprint-Ziel
Daily	Tägliche Synchronisation (max. 15 min)
Review	Ergebnisse zeigen, Feedback einholen
Retrospektive	Zusammenarbeit reflektieren & verbessern

Artefakt	Inhalt
Product Backlog	Alle User Stories, priorisiert
Sprint Backlog	Für die Iteration zugesagte Aufgaben

Artefakt	Inhalt
Increment	Lauffähiges, getestetes Ergebnis

Kanban-Board

Der Arbeitsfluss wurde auf einem Kanban-Board visualisiert. Aufgaben wandern nach dem Pull-Prinzip von links nach rechts; eine Aufgabe erreicht die Spalte „Done“ erst, wenn sie die Definition of Done erfüllt. Die Spalte „Review / Testing“ wurde im Projekt aktiv zur Nachverfolgung der Testfälle genutzt. WIP-Limits begrenzen die Anzahl paralleler Aufgaben pro Spalte und verhindern, dass zu viel gleichzeitig begonnen, aber nichts fertiggestellt wird.

Backlog	To Do (WIP 3)	In Progress (WIP 2)	Review / Testing	Done
US-033 KI-Quiz	US-014 Set forken	US-012 Sichtbarkeit	US-031 Streak	US-001 Registrierung
US-042 Freunde	US-022 JSON-Import		US-005 Profil	US-010 Set erstellen
US-024 Reihenfolge				US-030 Lernmodus

Definition of Ready & Definition of Done

Definition of Ready (DoR) — eine Story darf erst gezogen werden, wenn:

- Titel, Beschreibung & Akzeptanzkriterien vorhanden sind,
- der Nutzen klar formuliert ist,
- Schätzung & Priorität gesetzt sind.

Definition of Done (DoD) — eine Aufgabe gilt als erledigt, wenn:

- alle Akzeptanzkriterien erfüllt sind,
- der Code per Pull Request reviewt & nach main gemerged wurde,
- der Fall manuell bzw. per E2E-Test geprüft ist.

Aufwandsschätzung

Der Aufwand der User Stories wurde im Planning relativ in Story Points geschätzt. Als Methode diente Planning Poker: Alle Teammitglieder geben gleichzeitig eine Schätzung ab, wodurch gegenseitige Beeinflussung vermieden wird. Weichen die Schätzungen stark ab, deutet das auf ein unterschiedliches Verständnis der Aufgabe hin und wird im Team geklärt — die Diskussion ist dabei wertvoller als die exakte Zahl.

Iterationen im Projektverlauf

Iteration	Zeitraum	Schwerpunkt
Phase 1	19.05.2026	Grundfunktionen (Auth, Sets, Karten, Lernmodus, KI-Quiz)
Phase 2	26.05.2026	Erweiterungen (Fork, Tour, Profil, Streak, Dark-Mode)
Phase 3	26.05.–01.06.2026	Bugfixes & Verbesserungen
Phase 4	01.06.–09.06.2026	Soziale Funktionen, KI-Anbieter-Wechsel, UI-Feinschliff
Phase 5	23.06.2026	Sicherheits-Fix Passwort-Reset (OTP)

Git-Workflow

Jedes Teammitglied arbeitete auf einem eigenen Feature-Branch. Fertige Arbeit wurde per Pull Request vorgeschlagen, im Team reviewt und nach Freigabe in den Produktionsstand main integriert. Dieser Ablauf bildet die DoD technisch ab und hält main stets lauffähig.

3 Anforderungen & User Stories

Die Anforderungen wurden als User Stories aus Benutzerperspektive formuliert und bilden mit ihren Akzeptanzkriterien die Grundlage der Entwicklung.

Anforderungsarten & Story-Template

In der Requirements-Phase werden Anforderungen klassisch in funktionale (was das System leisten soll) und nicht-funktionale Anforderungen (Qualitätseigenschaften wie Sicherheit, Performance, Usability) unterschieden. StudyMate formuliert die funktionalen Anforderungen durchgängig als User Stories; zentrale nicht-funktionale Anforderungen sind u. a. Schutz vor Username-Enumeration, responsive Bedienbarkeit und ein konsistentes Design-System.

Feld	Inhalt
Titel	Kurzbezeichnung (z. B. „Registrierung“)
Beschreibung	„Als <Rolle> möchte ich <Ziel>, damit <Nutzen>.“
Akzeptanzkriterien	Prüfbare Bedingungen für „erledigt“
Schätzung & Priorität	Story Points und Reihenfolge im Backlog

Epic 1 — Benutzerverwaltung

US-001 — Registrierung

Als neuer Benutzer möchte ich mich mit Benutzernamen und Passwort registrieren können, damit ich ein persönliches Konto für meine Lernsets erhalte.

Akzeptanzkriterien:

- Benutzername mindestens 3 Zeichen lang
- Benutzername nur Buchstaben, Zahlen und _
- Passwort mindestens 6 Zeichen lang
- Bei bereits vergebenem Benutzernamen verständliche Fehlermeldung
- Nach erfolgreicher Registrierung automatischer Login und Weiterleitung zum Dashboard

Implementiert: 2026-05-19 — Supabase Auth-Integration

Datei: frontend/src/StudyMate.jsx (AuthView, handleRegister)

US-002 — Login

Als registrierter Benutzer möchte ich mich mit Benutzernamen und Passwort einloggen können, damit ich auf meine gespeicherten Lernsets zugreifen kann.

Akzeptanzkriterien:

- Login funktioniert mit korrektem Benutzernamen und Passwort
- Bei falschem Login benutzerfreundliche Fehlermeldung (kein technisches Detail)

- Sitzung bleibt bis zum manuellen Logout oder Token-Ablauf aktiv

Implementiert: 2026-05-19

Datei: frontend/src/StudyMate.jsx (handleLogin)

US-003 — Gastmodus

Als Besucher möchte ich die Plattform ohne Registrierung ausprobieren können, damit ich öffentliche Lernsets ansehen kann, bevor ich mich anmelde.

Akzeptanzkriterien:

- „Als Gast fortfahren“-Button auf der Login-Seite sichtbar
- Im Gastmodus nur öffentliche Sets sichtbar
- Aktionen wie Set-Erstellen leiten den Gast zur Anmeldeseite
- Gäste können Sets als Favoriten markieren; Migration ins Konto bei Anmeldung

Implementiert: 2026-05-19

Datei: frontend/src/StudyMate.jsx (handleGuest)

US-004 — Passwort zurücksetzen

Als Benutzer möchte ich mein vergessenes Passwort zurücksetzen können, damit ich wieder Zugriff auf mein Konto erhalte.

Akzeptanzkriterien:

- Über „Passwort vergessen?“ gelangt man zur Reset-Seite
- Eingabe des Benutzernamens sendet einen 6-stelligen Code an die Recovery-E-Mail
- Serverantwort verrät nicht, ob ein Benutzer existiert (Schutz vor Enumeration)
- Code ist 1 Stunde gültig und wird direkt im Formular eingegeben (kein Klick-Link)

Hinweis (2026-06-23): Ursprünglich wurde ein klickbarer Magic-Link verwendet. E-Mail-Sicherheitsscanner (z. B. Gmail/Antivirus) verbrauchten den Einweg-Token bereits beim Öffnen der Mail, bevor der Nutzer klickte — der Link landete dann ohne gültige Session auf der Startseite. Die Umstellung auf einen manuell eingegebenen OTP-Code behebt das, da kein automatisierter Request den Code mehr verbrauchen kann.

Implementiert: 2026-05-19; auf OTP-Code umgestellt: 2026-06-23

Datei: backend/app/routers/auth.py, frontend/src/components/AuthViews.jsx (ForgotPasswordView)

US-005 — Profilansicht und -bearbeitung

Als angemeldeter Benutzer möchte ich mein Profil mit Anzeigenamen, Bio und Profilbild verwalten können, damit andere Benutzer mehr über mich erfahren.

Akzeptanzkriterien:

- Profilseite zeigt Anzeigenname, Bio, Statistiken (Anzahl Sets, Karten, Streak-Tage)
- Anzeigenname und Bio bearbeitbar
- Profilbild hochladbar oder entfernbar

- Änderungen in Supabase (Anzeigename) und localStorage (Bio, Bild) gespeichert
- Profilbild erscheint als Avatar in NavBar und Sidebar-Fußzeile

Implementiert: 2026-05-26; Profilbild in NavBar/Sidebar: 2026-06-01

Datei: frontend/src/StudyMate.jsx (ProfileView, ProfileEditView, handleSaveProfile)

Epic 2 — Lernset-Verwaltung

US-010 — Lernset erstellen

Als angemeldeter Benutzer möchte ich ein neues Lernset mit Titel, Beschreibung und Sichtbarkeit erstellen können, damit ich meine Lernkarten strukturiert organisieren kann.

Akzeptanzkriterien:

- Dialog öffnet sich bei Klick auf „Neues Set“
- Titel ist Pflichtfeld; Beschreibung optional
- Sichtbarkeit zwischen „Privat“ und „Öffentlich“ wählbar
- Nach Erstellung direkt in der Set-Detailansicht

Implementiert: 2026-05-19

Datei: frontend/src/StudyMate.jsx (handleCreateSet, submitCreateSet)

US-011 — Lernset bearbeiten

Als Eigentümer eines Sets möchte ich Titel und Beschreibung des Sets nachträglich ändern können, damit ich Fehler korrigieren oder das Set umbenennen kann.

Akzeptanzkriterien:

- Bearbeitungs-Button neben dem Set-Titel (nur für Eigentümer)
- Titel und Beschreibung änderbar und speicherbar
- Änderungen sofort in der Ansicht reflektiert

Implementiert: 2026-05-26 (Commit 9282ff3)

Datei: frontend/src/StudyMate.jsx (saveEditTitle, handleUpdateSetTitle)

US-012 — Sichtbarkeit umschalten

Als Eigentümer möchte ich ein Set von privat auf öffentlich (und zurück) schalten können, damit ich entscheide, wer meine Sets sehen kann.

Akzeptanzkriterien:

- Umschalten nur für Eigentümer
- Bestätigungsdiallog vor der Änderung
- Badge zeigt aktuellen Status (Öffentlich/Privat)
- Beim Veröffentlichen fragt ein Dialog, ob der Ersteller (Avatar/Name) sichtbar sein soll (show_author); Standard sichtbar

Implementiert: 2026-05-19; Autoren-Sichtbarkeits-Dialog: 2026-06-02

Datei: frontend/src/StudyMate.jsx (handleToggleSetVisibility),
components/DetailView.jsx (showAuthorDialog)

US-013 — Lernset löschen

Als Eigentümer möchte ich ein Set endgültig löschen können, wenn ich es nicht mehr benötige.

Akzeptanzkriterien:

- Löschen nur für Eigentümer
- Bestätigungsdialog vor dem Löschen
- Nach dem Löschen Rückkehr zur Dashboard-Ansicht

Implementiert: 2026-05-19

Datei: frontend/src/StudyMate.jsx (handleDeleteSet)

US-014 — Öffentliches Set forken / kopieren

Als angemeldeter Benutzer möchte ich ein öffentliches Set in mein Konto kopieren (forken) können, damit ich es bearbeiten und anpassen kann.

Akzeptanzkriterien:

- Fork-Button bei öffentlichen, fremden Sets (nicht für Gäste)
- „Set kopieren“-Button für eigene Sets (öffentlich wie privat)
- Dialog erlaubt Umbenennen vor dem Speichern
- Geforktes/kopiertes Set ist zunächst privat
- Alle Karten des Original-Sets werden übernommen

Implementiert: 2026-05-26 (Commits 98d6cdf, 4a41540, e46744c); Priv-zu-Priv: 2026-06-02

Datei: backend/app/routers/sets.py (fork_set), frontend/src/StudyMate.jsx (handleForkSet, submitForkSet)

Epic 3 — Karteikarten-Verwaltung

US-020 — Karteikarte hinzufügen

Als Eigentümer eines Sets möchte ich neue Karteikarten mit Frage und Antwort hinzufügen können, damit das Set wächst.

Akzeptanzkriterien:

- Formular für Frage und Antwort in der Detailansicht
- Karte erscheint sofort nach dem Speichern in der Liste
- Fehler werden bei leeren Feldern angezeigt

Implementiert: 2026-05-19

Datei: frontend/src/StudyMate.jsx (addCard, handleAddCard)

US-021 — Karteikarte bearbeiten

Als Eigentümer möchte ich bestehende Karteikarten bearbeiten können, damit ich Fehler in Fragen oder Antworten korrigieren kann.

Akzeptanzkriterien:

- Bearbeiten-Button bei jeder Karte (nur für Eigentümer)
- Inline-Formular mit vorausgefüllten Feldern
- Änderungen sofort in State und UI reflektiert

Implementiert: 2026-05-19 (Commit c2b967a)

Datei: frontend/src/StudyMate.jsx (openEditCard, saveEditCard, handleEditCard)

US-022 — Karteikarten per JSON importieren

Als Benutzer möchte ich mehrere Karteikarten auf einmal per JSON importieren können, damit ich schnell große Lernsets aufbauen kann.

Akzeptanzkriterien:

- Import-Funktion in der Detailansicht (nur für Eigentümer)
- JSON-Format: Array von Objekten mit question und answer
- Ungültige Eingaben erzeugen verständliche Fehlermeldung
- Nach Erfolg werden importierte Karten sofort angezeigt

Implementiert: 2026-05-26 (Commit 9282ff3)

Datei: frontend/src/StudyMate.jsx (importCards, handleImportCards)

US-023 — Karteikarte löschen

Als Eigentümer möchte ich einzelne Karteikarten löschen können, damit ich veraltete oder fehlerhafte Karten entfernen kann.

Akzeptanzkriterien:

- Löschen-Button bei jeder Karte (nur für Eigentümer)
- Bestätigungsdialo vor dem Löschen

- Karte verschwindet sofort aus der Ansicht

Implementiert: 2026-05-19

Datei: frontend/src/StudyMate.jsx (deleteCard, handleDeleteCard)

US-024 — Karten-Reihenfolge ändern

Als Eigentümer eines Sets möchte ich die Reihenfolge der Karteikarten anpassen können, damit ich thematisch verwandte Karten gruppieren kann.

Akzeptanzkriterien:

- Pfeil-Buttons (▲/▼) neben jeder Karte (nur für Eigentümer)
- Karte tauscht Position mit dem Nachbar oberhalb/unterhalb
- Neue Reihenfolge dauerhaft in der Datenbank (position-Feld)
- Lern- und Quizmodus verwenden die aktualisierte Reihenfolge

Implementiert: 2026-06-02

Datei: frontend/src/StudyMate.jsx (handleMoveCard), components/DetailView.jsx (ChevronUp/ChevronDown)

Epic 4 — Lernen und Quiz

US-030 — Lernmodus (Flip-Card)

Als Benutzer möchte ich meine Karteikarten im Flip-Card-Format durchgehen können, damit ich den Lernfortschritt tracken und mich selbst testen kann.

Akzeptanzkriterien:

- Karten nacheinander angezeigt (Vorderseite = Frage)
- Klick dreht die Karte und zeigt die Antwort
- Nach Aufdecken „Gewusst“ oder „Nicht gewusst“ angebbbar
- Fortschrittsbalken zeigt aktuelle Position
- Am Ende Zusammenfassung mit gewussten/nicht-gewussten Karten
- Option, die Session zu wiederholen

Implementiert: 2026-05-19

Datei: frontend/src/StudyMate.jsx (LearnView)

US-031 — Streak-Tracking

Als Benutzer möchte ich meinen täglichen Lern-Streak sehen können, damit ich motiviert bleibe, jeden Tag zu lernen.

Akzeptanzkriterien:

- Streak im Dashboard und im Profil angezeigt
- Streak erhöht sich nur einmal pro Kalendertag nach abgeschlossener Session
- Bei Pause an einem Tag Reset auf 0
- Daten in localStorage gespeichert (persistiert über Sessions)

Implementiert: 2026-05-26 (Commit 28cf558)

Datei: frontend/src/StudyMate.jsx (awardDailyStreak, getStreakState, toISODate, getDaysSince)

US-032 — Standard-Quiz

Als Benutzer möchte ich ein Multiple-Choice-Quiz aus meinen Karteikarten starten können, damit ich mein Wissen testen kann.

Akzeptanzkriterien:

- Quiz verfügbar ab mindestens 4 Karten
- Falsche Antworten aus anderen Karten des Sets generiert (Distraktoren)
- Nach Auswahl richtige Antwort grün, falsche rot markiert
- Am Ende Auswertung (richtig/falsch)

Implementiert: 2026-05-19

Datei: frontend/src/StudyMate.jsx (QuizView, Phase quiz)

US-033 — KI-Quiz

Als Benutzer möchte ich ein KI-generiertes Quiz starten können, damit ich mit neuartigen, thematisch passenden Fragen geübt werde.

Akzeptanzkriterien:

- KI-Quiz über Backend-Endpoint /quiz/generate generiert
- Jede Frage: 4 Antwortoptionen (3 falsche, 1 richtige) + Erklärung
- Nach der Antwort wird die Erklärung angezeigt
- Ohne Groq-API-Key Hinweismeldung (HTTP 503)

Implementiert: 2026-05-19 (Commit db4dcbb, initial Google Gemini); auf Groq/Llama umgestellt: 2026-06-01 (Commit 5e42b21)

Datei: backend/app/routers/quiz.py, frontend/src/StudyMate.jsx (generateAIQuiz, Phase aiquiz)

Epic 5 — Entdecken und soziale Funktionen

US-040 — Öffentliche Sets entdecken

Als Benutzer möchte ich öffentliche Sets anderer Benutzer entdecken und durchsuchen können, damit ich von der Lernarbeit anderer profitieren kann.

Akzeptanzkriterien:

- „Entdecken“-Tab im Dashboard zeigt alle öffentlichen Sets
- Suchfeld filtert Sets nach Titel in Echtzeit
- Vorgeschlagene Sets am Anfang der Liste

Implementiert: 2026-05-19

Datei: frontend/src/StudyMate.jsx (DashboardView, Tab discover)

US-041 — Favoriten

Als Benutzer möchte ich Sets als Favoriten markieren können, damit ich schnell auf häufig genutzte Sets zugreifen kann.

Akzeptanzkriterien:

- Stern-Button bei jedem Set (aktiv = gelb)
- Favoriten in localStorage (sm_favs_{userId} bzw. sm_favs_guest)
- Favoritenansicht zeigt nur markierte Sets (auch für Gäste)
- Beim Einloggen werden Gastfavoriten automatisch zusammengeführt

Implementiert: 2026-05-19; Gastfavoriten-Migration: 2026-06-02

Datei: frontend/src/StudyMate.jsx (toggleFavorite, FavoritesView)

US-042 — Freunde finden und verwalten

Als Benutzer möchte ich andere Benutzer suchen und als Freund hinzufügen können, damit ich mich mit ihnen vernetzen kann.

Akzeptanzkriterien:

- Suche nach Benutzername oder Anzeigename
- Freundschaftsanfrage senden, annehmen oder ablehnen
- Bestehende Freundschaften entfernbar
- Eingehende Anfragen in eigenem Tab mit Badge-Zähler in der Sidebar

Implementiert: 2026-06-02 (Commit a15800c)

Datei: frontend/src/components/FriendsView.jsx, StudyMate.jsx
(handleSendFriendRequest, handleAcceptFriend, handleDeclineFriend, handleRemoveFriend, handleSearchUsers)

US-043 — Öffentliches Profil ansehen

Als Benutzer möchte ich das öffentliche Profil eines anderen Benutzers ansehen können, damit ich seine öffentlichen Sets entdecken und ihn als Freund hinzufügen kann.

Akzeptanzkriterien:

- Aufrufbar über den Autorennamen eines öffentlichen Sets (bei show_author) oder aus der Freundesliste
- Zeigt Avatar, Anzeigename, Bio und öffentliche Sets
- Zeigt Freundschaftsstatus mit passender Aktion (Senden/Annehmen/Ablehnen/Entfernen)
- Zurück-Button führt zur ursprünglichen Ansicht zurück

Implementiert: 2026-06-02 (Commit a15800c)

Datei: frontend/src/components/PublicProfileView.jsx, StudyMate.jsx
(handleOpenUserProfile)

Epic 6 — Onboarding und UX

US-050 — Geführte Tour

Als neuer Benutzer möchte ich beim ersten Login eine geführte Tour durch die wichtigsten Funktionen erhalten, damit ich mich schnell zurechtfinde.

Akzeptanzkriterien:

- Autostart beim ersten Dashboard-Besuch nach Login
- Autostart beim ersten Öffnen einer Set-Detailansicht
- Autostart beim ersten Öffnen des „Set erstellen“-Dialogs
- Jeder Schritt hebt das relevante UI-Element hervor und erklärt es
- Tour jederzeit überspringbar
- Abgeschlossene Tours in localStorage (kein erneuter Autostart)
- Hilfe-Button in der NavBar ermöglicht manuelles Neustarten

Implementiert: 2026-05-26 (Commit 322ab03); Detail- & Erstellungs-Tour: 2026-06-02

Datei: frontend/src/StudyMate.jsx (GuidedTourOverlay, TOUR_STEPS)

US-051 — Light/Dark-Mode

Als Benutzer möchte ich zwischen einem hellen und einem dunklen Farbschema wechseln können, damit ich je nach Umgebung komfortabel lernen kann.

Akzeptanzkriterien:

- Umschalter in der Navigationsleiste
- Gewähltes Theme in localStorage gespeichert und beim nächsten Besuch wiederhergestellt
- Alle UI-Elemente passen sich dem Theme an

Implementiert: 2026-05-26 (Commit dcc0679)

Datei: frontend/src/StudyMate.jsx (State theme, CSS-Klasse .sm.light)

4 Architektur & Technologie-Stack

StudyMate folgt einer klassischen Drei-Schichten-Architektur, die eine klare Trennung zwischen Präsentation, Anwendungslogik und Datenhaltung herstellt. Die oberste Schicht ist ein React-Single-Page-Application-Client, der im Browser läuft und die gesamte Benutzeroberfläche rendert. Darunter liegt ein FastAPI-Backend, das als REST-Schnittstelle dient, eingehende Anfragen validiert, Berechtigungen prüft und die Geschäftslogik kapselt. Die unterste Schicht bildet Supabase (PostgreSQL) als Persistenz- und Authentifizierungsdienst. Externe Dienste — die Groq-API für die KI-Quiz-Generierung und die Resend-API für den E-Mail-Versand — werden ausschließlich serverseitig angesprochen, sodass keine sensiblen Schlüssel im Browser landen.

Diese Schichtung erlaubt es, einzelne Bestandteile unabhängig voneinander weiterzuentwickeln oder auszutauschen: Das Frontend kann ohne Änderung am Backend umgestaltet werden, und der KI-Anbieter lässt sich (wie in Phase 4 geschehen) hinter der Backend-Schnittstelle wechseln, ohne dass der Client etwas davon merkt.

Architekturprinzip: Trennung der Belange (MVC)

Die Architektur orientiert sich am Grundgedanken des MVC-Musters (Model-View-Controller): Daten, Darstellung und Steuerung werden in getrennte Verantwortlichkeiten aufgeteilt. Diese Trennung vereinfacht Wartung, Erweiterung und Testbarkeit, da Änderungen an einer Verantwortlichkeit die anderen möglichst wenig berühren. Die folgende Tabelle zeigt, wie sich die drei MVC-Rollen konkret auf die Bausteine von StudyMate abbilden:

MVC-Rolle	Umsetzung in StudyMate
Model	Supabase/PostgreSQL als Datenhaltung; Pydantic-Schemas als Datenmodell im Backend
View	React-Komponenten (DashboardView, DetailView, LearnView, ...)
Controller	FastAPI-Router (Steuerlogik, Validierung) und die Event-Handler im Frontend

Schichtenarchitektur

Das folgende Diagramm zeigt die drei Schichten und ihre Kommunikationswege. Der Client spricht ausschließlich das Backend per REST an; das Backend kapselt sämtliche Zugriffe auf die Datenbank und die externen Dienste.

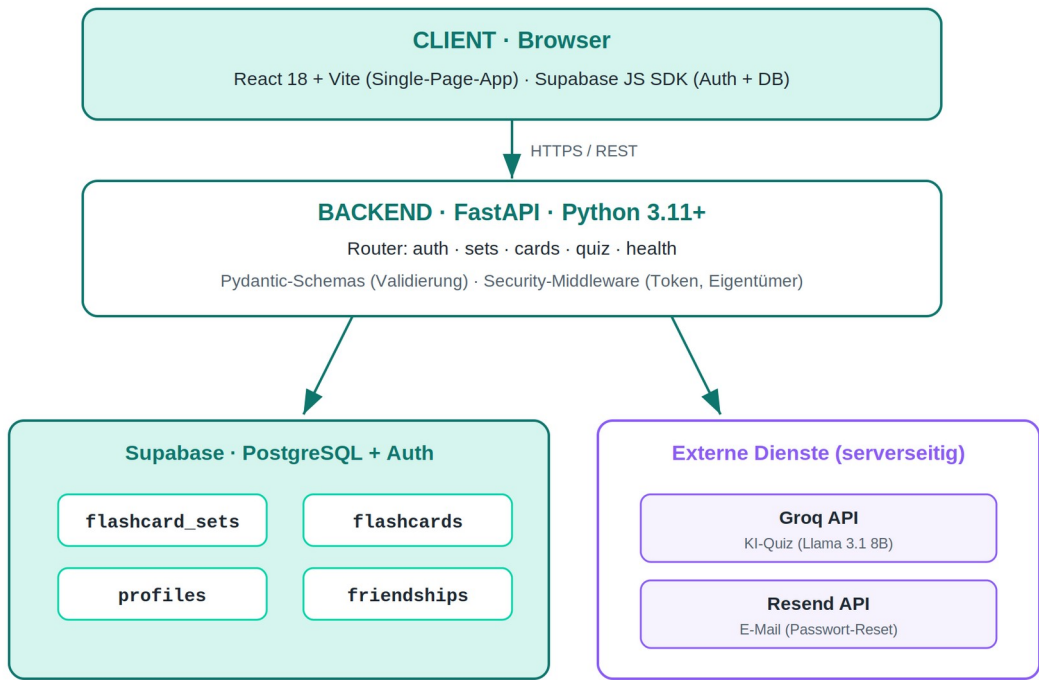


Abb. 1: Drei-Schichten-Architektur von StudyMate mit Datenbank und serverseitigen Fremddiensten.

Technologie-Stack

Schicht	Technologie	Anmerkung
Frontend-Framework	React 18	Single-Page-Application
Build-Tool	Vite	Dev-Server & Production-Build
Styling	Inline CSS + dynamische Styles	injiziert in StudyMate.jsx
Icons	Lucide React	—
Backend-Framework	FastAPI	REST-API
Sprache (Backend)	Python 3.11+	—
Validierung	Pydantic v2	typsichere Schemas
Datenbank & Auth	Supabase (PostgreSQL)	inkl. Auth-Service
E-Mail	Resend API	Passwort-Reset
KI-Integration	Groq (Llama 3.1 8B Instant)	OpenAI-kompatible API
Deployment	Vercel	Frontend & Backend

Projektstruktur (vollständig)

Das Repository ist klar in Frontend, Backend und Dokumentation gegliedert. Der folgende Verzeichnisbaum zeigt alle relevanten Dateien mit ihrer jeweiligen Aufgabe:

```
studymate_v2/  
├─ documentation/
```

└ full_documentation_de.md	← Diese Datei
└ frontend/	
└ src/	
└ StudyMate.jsx	← Gesamte App-Logik und alle Views
└ main.jsx	← React-Einstiegspunkt
└ components/	← DetailView, FriendsView, PublicProfileView, AuthViews, Sidebar, Avatar
└ supabase.js	← Supabase-Client-Konfiguration
└ e2e/	← Playwright-E2E-Tests (auth, sets, cards, ui,
mobile)	
└ index.html	
└ package.json	
└ vite.config.js	
└ playwright.config.js	
└ vercel.json	
└ backend/	
└ app/	
└ main.py	← FastAPI App-Initialisierung
└ core/	
└ config.py	← Pydantic Settings
└ security.py	← Token-Validierung, Eigentümer-Prüfung
└ db/	
└ supabase_client.py	← Supabase Admin/Public Clients
└ routers/	
└ auth.py	← Passwort-Reset-Endpunkt
└ sets.py	← CRUD Lernsets + Fork
└ cards.py	← CRUD Karteikarten
└ quiz.py	← KI-Quiz-Generierung
└ health.py	← Health-Check
└ schemas/	
└ sets.py	← StudySetCreate, StudySetUpdate
└ cards.py	← FlashcardCreate
└ quiz.py	← QuizGenerateRequest/Response
└ auth.py	← ForgotPasswordRequest
└ app.py	← Uvicorn-Einstiegspunkt
└ requirements.txt	
└ sql_schema.sql	← Referenz-SQL für Tabellen

5 Datenbankmodell

Die Persistenz erfolgt über Supabase (PostgreSQL). Das Datenmodell besteht aus vier Kerntabellen. `flashcard_sets` und `flashcards` werden über das FastAPI-Backend angesprochen, `profiles` und `friendships` hingegen direkt über den Supabase JS-Client im Frontend. Alle Benutzer referenzieren die von Supabase verwaltete Tabelle `auth.users`.

Entity-Relationship-Modell

Das folgende ER-Diagramm zeigt die vier Kerntabellen samt der von Supabase verwalteten `auth.users` sowie die Beziehungen zwischen ihnen (Primärschlüssel PK, Fremdschlüssel FK). Ein Benutzer besitzt beliebig viele Lernsets, jedes Lernset enthält beliebig viele Karteikarten, und über `forked_from` referenziert ein geforktes Set sein Quell-Set.

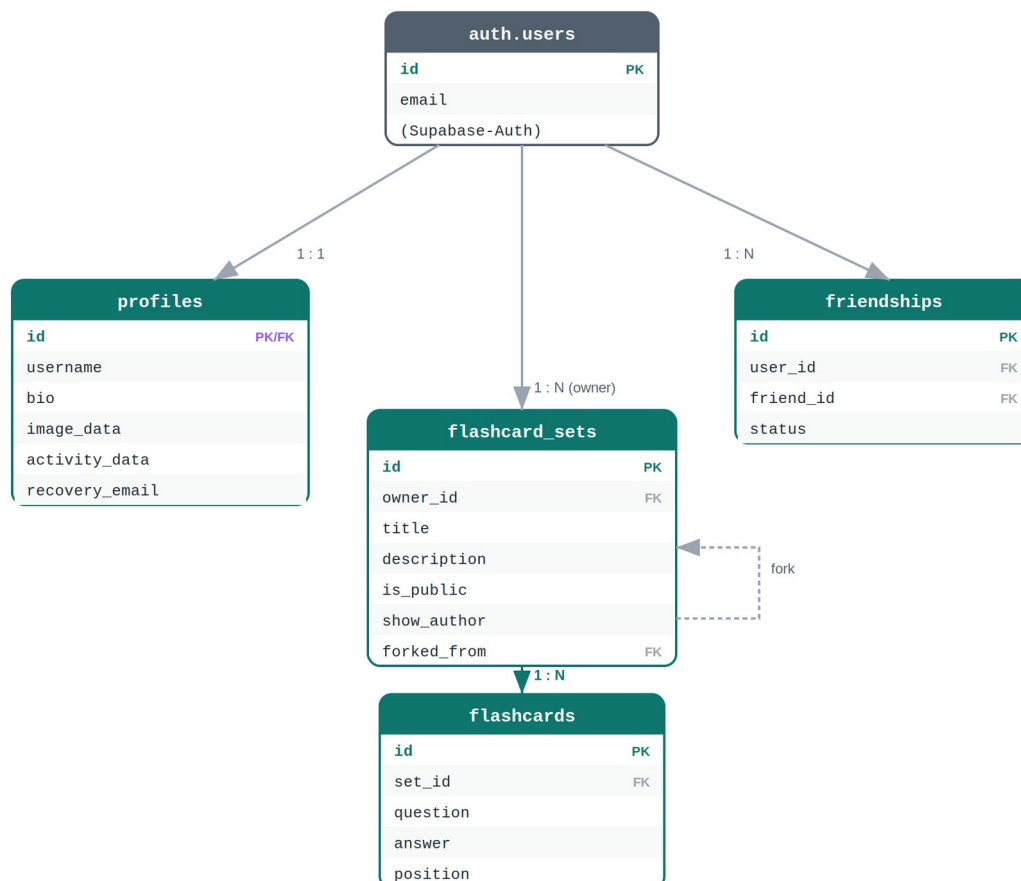


Abb. 2: Entity-Relationship-Modell der StudyMate-Datenbank mit Primär- und Fremdschlüsselbeziehungen.

Datenzugriffspfade

StudyMate nutzt bewusst zwei unterschiedliche Zugriffswege auf die Datenbank: Lernsets und Karteikarten laufen über das FastAPI-Backend, das die Eigentümer-Prüfung übernimmt. Profile und Freundschaften werden dagegen direkt über den Supabase JS-Client aus dem Frontend gelesen und geschrieben — ohne eigenen Backend-Endpunkt.

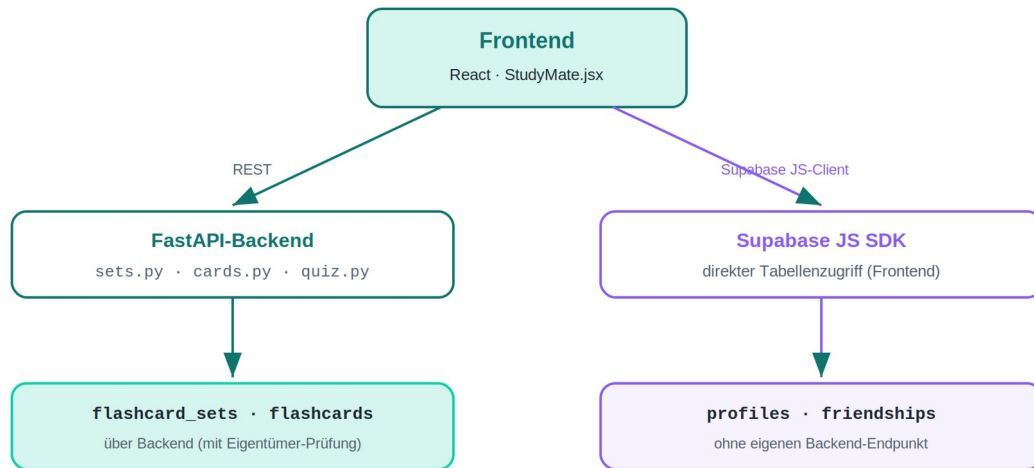


Abb. 3: Die zwei Datenzugriffspfade — über das Backend (mit Eigentümer-Prüfung) bzw. direkt über den Supabase JS-Client.

Tabellen im Detail

flashcard_sets — Lernsets

Spalte	Typ	Beschreibung
id	uuid (PK)	Eindeutige ID, automatisch generiert
owneruserid	uuid (FK)	Eigentümer → auth.users
title	text	Titel (Pflichtfeld)
description	text	Beschreibung (optional)
ispublic	bool	Öffentlich oder privat (Standard: false)
createdat / updatedat	timestampz	Erstellungs- / Änderungszeitpunkt
forked_from	uuid (FK)	Quell-Set bei Forks (self-reference, optional)
show_author	bool	Ersteller bei öffentlichen Sets anzeigen (Standard: true)

flashcards — Karteikarten

Spalte	Typ	Beschreibung
id	uuid (PK)	Eindeutige ID
setid	uuid (FK)	Zugehöriges Set → flashcard_sets
question / answer	text	Frage / Antwort (beide Pflicht)
position	int4	Sortierschlüssel für Lern-/Quiz-Modus
createdat / updatedat	timestampz	Zeitstempel

profiles — Benutzerprofile

Spalte	Typ	Beschreibung
id	uuid (PK/FK)	Supabase Auth User ID
username	text (unique)	Benutzername (lowercase, eindeutig)
displayname	text	Anzeigename
recovery_email	text	E-Mail für Passwort-Reset (optional)
bio / image_data	text	Kurzbeschreibung / Profilbild (Base64, optional)
streak_count	int4	Aktueller Lern-Streak (Tage in Folge)
streak_last_date	text	Datum (ISO) des letzten Streak-Updates
activity_data	jsonb	Tägliche Lernaktivität (WeeklyActivityChart)

friendships — Freundschaften

Spalte	Typ	Beschreibung
id	uuid (PK)	Eindeutige ID

Spalte	Typ	Beschreibung
user_id	uuid (FK)	Anfragender Benutzer
friend_id	uuid (FK)	Angefragter Benutzer
status	text	pending (Anfrage offen) oder accepted (befreundet)
created_at	timestampz	Zeitpunkt der Anfrage

Hinweis: profiles und friendships werden ausschließlich direkt über den Supabase JS-Client im Frontend gelesen/geschrieben (kein eigener Backend-Endpunkt) — anders als flashcard_sets / flashcards, die über die FastAPI-Routen sets.py / cards.py laufen.

SQL-Referenzschema (aus sql_schema.sql)

```
CREATE TABLE IF NOT EXISTS public.study_sets (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  owner_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  description TEXT,
  is_public BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS public.flashcards (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  set_id UUID NOT NULL REFERENCES public.study_sets(id) ON DELETE CASCADE,
  question TEXT NOT NULL,
  answer TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Schema-Pflege: In der produktiven Supabase-Datenbank weichen einige Spaltennamen vom Referenz-SQL ab (z. B. owneruserid statt owner_id, ispublic statt is_public). Die Tabellen profiles / friendships sowie forked_from und show_author fehlen aktuell noch in sql_schema.sql — die Synchronisierung ist als offene Aufgabe vermerkt (siehe Kapitel 16).

6 Backend — Komponenten & API-Referenz

Das Backend ist eine FastAPI-Anwendung. `main.py` erstellt die App, registriert die CORS-Middleware und bindet die Router ein. Geschützte Endpunkte validieren einen Bearer-Token über Supabase und prüfen die Eigentümerschaft.

6.1 `app/main.py` — Anwendungsinitialisierung

```
app = FastAPI(title="StudyMate API", version="0.1.0")
app.add_middleware(CORSMiddleware, allow_origins=["*"], ...)
app.include_router(health.router)
app.include_router(auth.router, prefix="/auth")
app.include_router(sets.router, prefix="/sets")
app.include_router(cards.router)
app.include_router(quiz.router)
```

6.2 `app/core/config.py` — Konfiguration

Verwendet `pydantic-settings` für typsichere Konfiguration aus Umgebungsvariablen.

```
class Settings(BaseSettings):
    supabase_url: str = ""
    supabase_anon_key: str = ""
    supabase_service_role_key: str = ""
    resend_api_key: str = ""
    resend_from: str = "StudyMate <onboarding@resend.dev>"
    groq_api_key: str = ""
```

6.3 `app/core/security.py` — Authentifizierung & Autorisierung

Funktion	Beschreibung
<code>get_current_user_id(token)</code>	Validiert Bearer-Token via Supabase, gibt User-ID zurück
<code>get_current_user(token)</code>	Wie oben, liefert vollständiges User-Objekt
<code>require_set_owner(set_id, user_id)</code>	Prüft Eigentümerschaft am Set (sonst HTTP 403)
<code>require_set_access(set_id, user_id)</code>	Zugriff, wenn Set öffentlich oder User Eigentümer
<code>require_card_owner(card_id, user_id)</code>	Prüft Eigentümerschaft über das Set der Karte

6.4 API-Endpunkte — Vollständige Referenz

Methode	Pfad	Auth	Beschreibung
POST	<code>/auth/forgot-password</code>	Nein	Sendet 6-stelligen Reset-Code per E-Mail
GET	<code>/sets/public</code>	Nein	Alle öffentlichen Sets
GET	<code>/sets/my</code>	Ja	Eigene Sets des Benutzers
POST	<code>/sets</code>	Ja	Neues Set erstellen

Methode	Pfad	Auth	Beschreibung
PUT	/sets/{set_id}	Eigentümer	Set aktualisieren
DELETE	/sets/{set_id}	Eigentümer	Set löschen
POST	/sets/{set_id}/fork	Ja	Set forken (Kopie wird privat)
GET	/sets/{set_id}/cards	Optional	Karten eines Sets (öffentlich ohne Auth)
POST	/sets/{set_id}/cards	Eigentümer	Neue Karte erstellen
DELETE	/cards/{card_id}	Eigentümer	Karte löschen
POST	/quiz/generate	Nein	KI-Quiz generieren (Groq/Llama)
GET	/health	Nein	Health-Check

Passwort-Reset — POST /auth/forgot-password

Response: { "message": "ok" } (immer, unabhängig davon, ob der Benutzer existiert — Schutz vor Username-Enumeration). Der Backend-Router sucht die Recovery-E-Mail aus der profiles-Tabelle. Falls vorhanden, wird über die Supabase Admin API (generate_link, Typ recovery) ein email_otp-Code erzeugt und via Resend versandt. Das Frontend ruft anschließend supabase.auth.verifyOtp({ email, token, type: "recovery" }) auf, um eine Session zu erhalten, und setzt das neue Passwort via supabase.auth.updateUser(). Es gibt keinen klickbaren Link und keinen Redirect — der Code wird manuell im Formular eingegeben.

Fork-Logik — POST /sets/{set_id}/fork

- Prüft, ob das Quell-Set zugänglich ist; private fremde Sets → HTTP 403.
- Erstellt eine Kopie mit ispublic = false und übernimmt alle Karten.
- Karten werden nach position abgefragt und mit korrektem position-Feld kopiert, damit die Reihenfolge erhalten bleibt (Bugfix 06/2026).

Request-Schema StudySetCreate:

```
{
  "title": "Cybersecurity Grundlagen",
  "description": "Wichtige Begriffe und Konzepte",
  "is_public": false
}
```

Karten — GET /sets/{set_id}/cards

- Öffentliche Sets sind ohne Auth abrufbar.
- Private Sets erfordern gültigen Bearer-Token des Eigentümers.
- HTTP 401 bei fehlendem Token, HTTP 403 bei falschem Benutzer.

Request-Schema FlashcardCreate:

```
{
  "question": "Was ist SQL Injection?",
  "answer": "Eine Angriffstechnik, bei der SQL-Code eingeschleust wird.",
}
```

```
"position": 1
}
```

KI-Quiz — POST /quiz/generate

Die KI (Groq, Modell llama-3.1-8b-instant über die OpenAI-kompatible API unter <https://api.groq.com/openai/v1>) generiert ausschließlich die drei falschen Antwortoptionen sowie eine Erklärung. Die korrekte Antwort stammt direkt aus der Datenbank und wird an zufälliger Position eingefügt. Ohne konfigurierten GROQ_API_KEY antwortet der Endpunkt mit HTTP 503.

Request-Schema QuizGenerateRequest:

```
{
  "cards": [
    { "q": "Was ist Phishing?", "a": "Betrugsversuch per gefälschten E-Mails" }
  ],
  "count": 5
}
```

Response-Schema QuizGenerateResponse:

```
{
  "questions": [
    {
      "question": "Was ist Phishing?",
      "options": ["Option A", "Option B", "Betrugsversuch ...", "Option D"],
      "correct": 2,
      "explanation": "Phishing bezeichnet den Versuch, über gefälschte ..."
    }
  ]
}
```

7 Frontend — Komponenten & Funktionen

Die gesamte Frontend-Logik ist bewusst zentral in frontend/src/StudyMate.jsx gebündelt, um die Projektkonfiguration schlank zu halten; größere Teilansichten (DetailView, FriendsView, PublicProfileView, AuthViews, Sidebar) sind in components/ ausgelagert.

7.1 Hilfsfunktionen

Funktion	Beschreibung	Implementiert
accentFor(id)	Deterministischer Farbbhash aus der Set-ID	2026-05-19
normalizeSet(raw)	Transformiert Supabase-Rohdaten in App-Format	2026-05-19
toISODate(date)	Konvertiert Date-Objekt in YYYY-MM-DD	2026-05-26
parseISODate(str)	Parst YYYY-MM-DD sicher zu Date	2026-05-26
getDaysSince(str)	Berechnet Tage seit einem Datum	2026-05-26
readStreakStore()	Liest Streak-Daten aus localStorage	2026-05-26
writeStreakStore(store)	Schreibt Streak-Daten in localStorage	2026-05-26
getStreakState(key)	Gibt aktuellen Streak zurück; Reset bei >1 Tag Pause	2026-05-26
awardDailyStreak(key)	Erhöht Streak einmalig pro Kalendertag	2026-05-26
readProfileSettings(id)	Liest Profildetails (Bio, Bild) aus localStorage	2026-05-26
writeProfileSettings(id, s)	Schreibt Profildetails in localStorage	2026-05-26

7.2 Navigations-Zustände (view)

View	Beschreibung	Komponente
auth	Login / Registrierung	AuthView
forgot	Passwort-Reset (Code + neues Passwort)	ForgotPasswordView
dashboard	Hauptansicht mit Tabs & Statistiken	DashboardView
detail	Set-Detailansicht (Karten, Aktionen)	DetailView
learn	Lernmodus	LearnView
quiz	Quiz-Modus	QuizView
favorites	Favoritenansicht	FavoritesView
friends	Freundesliste, Anfragen, Suche	FriendsView
public_profile	Öffentliches Profil eines Nutzers	PublicProfileView
profile	Profilansicht	ProfileView
profile_edit	Profil bearbeiten	ProfileEditView
settings	Einstellungen (Platzhalter)	SettingsView

7.3 Hauptkomponenten

NavBar — Zeigt Logo, Tour-Button, Theme-Umschalter, Profil-Avatar und Logout-Button.

Props: user, onHome, onLogout, onGoToLogin, theme, onToggleTheme, onProfile, onTourRestart, currentViewHasTour. Zeigt im Avatar-Button das hochgeladene Profilbild (user.imageData) als — fällt auf den Anfangsbuchstaben zurück, wenn kein Bild vorhanden ist. Implementiert: 2026-05-19; Tour-Button: 2026-05-26; Profilbild-Anzeige: 2026-06-01 (cf603c0).

Sidebar — Kollabierbare Navigation (Dashboard, Meine Sets, Entdecken, Favoriten, Freunde); auf Mobilgeräten als Overlay, auf Desktop als feste Leiste. Die Fußzeile zeigt seit 2026-06-01 einen kleinen Avatar neben dem Benutzernamen — konsistent mit dem NavBar-Avatar. Eigenständige Komponente seit 2026-06-01 (a52e919).

DashboardView — Drei Tabs (Dashboard/Entdecken/Meine Sets), Statistikkarten (Sets, Karten, Streak), Vorschlagskarussell und WeeklyActivityChart (Lernaktivität der letzten 7 Tage aus profiles.activity_data). Bei aktiviertem show_author wird der Ersteller mit Avatar/Name angezeigt und ist anklickbar (öffnet public_profile). WeeklyActivityChart: 2026-06-01, responsives Feintuning: 2026-06-02.

DetailView — Karten verwalten (hinzufügen, bearbeiten, löschen, JSON-Import), Set-Titel/-Beschreibung bearbeiten, Sichtbarkeit umschalten, Set löschen, Fork-Button bei fremden öffentlichen Sets.

Verhaltensänderung (cf603c0, 2026-06-01): Die „Lernen starten“- und „Quiz starten“-Buttons sind nicht mehr bei leeren Sets deaktiviert. Die zugehörige Hinweismeldung wurde entfernt; der LearnView-Guard für leere Sets entfiel ebenfalls.

LearnView — Flip-Card mit CSS-3D-Transformation. Benutzer markiert jede Karte als gewusst/nicht gewusst; am Ende Zusammenfassung + Wiederholen. Beim Abschließen wird handleCompleteSet() aufgerufen, der den Streak aktualisiert.

QuizView — Standard-Quiz (Multiple-Choice aus vorhandenen Karten, mind. 4) und KI-Quiz (Groq/Llama 3.1) mit Erklärung nach jeder Antwort.

GuidedTourOverlay — Overlay-Komponente, die UI-Elemente per CSS-Highlight hervorhebt und Erklärungen in einem Popup anzeigt. Schritte sind in TOUR_STEPS pro View definiert. Tour-Status wird in localStorage unter sm_tour_completed gespeichert.

```
const TOUR_STEPS = {
  dashboard: [...], // 4 Schritte
  discover:  [...], // 3 Schritte
  mine:      [...], // 2 Schritte
  detail:    [...], // 3 Schritte
  createSet: [...], // 2 Schritte
  learn:     [...], // 4 Schritte
  quiz:      [...], // 3 Schritte
  favorites: [...], // 3 Schritte
  friends:   [...], // 2 Schritte
  profile:   [...], // 2 Schritte
```

```
};
```

ProfileView / ProfileEditView — Profilansicht mit Statistiken bzw. Formular für Anzeigenname, Bio und Profilbild (Base64). Änderungen werden in Supabase (profiles.displayname, .bio, .image_data) gespeichert. Aus einer früheren rein lokalen Version existiert noch eine einmalige Migration: Beim Login werden alte localStorage-Profildaten in die Supabase-Spalten übernommen, falls dort noch nichts hinterlegt ist.

FriendsView / PublicProfileView — Soziale Funktionen direkt über den Supabase-Client (Tabelle friendships, kein Backend-Endpunkt): Freunde, eingehende Anfragen mit Annehmen/Ablehnen, Benutzersuche; öffentliches Profil mit Avatar, Bio, öffentlichen Sets und Freundschaftsstatus.

8 Authentifizierung & Sicherheit

Auth-Mechanismus

1. Frontend-Auth: Das Supabase JS-SDK verwaltet Sessions. Der Benutzername wird in eine Fake-E-Mail `username@studymate.local` übersetzt, da Supabase ein E-Mail-Format erwartet.

```
const toFakeEmail = (username) => `${username.toLowerCase()}@studymate.local`;
```

2. Backend-Auth: Geschützte Endpunkte erwarten einen Bearer-Token im Authorization-Header, der über den Supabase Public Client validiert wird.
3. Session-Management: `onAuthStateChange` wird vor `getSession()` registriert, um Login/Logout-Events korrekt abzufangen.
4. Passwort-Reset: Läuft seit 2026-06-23 über `supabase.auth.verifyOtp()` (manuelle Code-Eingabe) statt über einen Redirect-Link — kein URL-/Hash-Parsing beim App-Start mehr nötig.

Sicherheitsrichtlinien

Aspekt	Maßnahme
Username-Enumeration	Passwort-Reset gibt immer { "message": "ok" } zurück
Service Role Key	Ausschließlich im Backend, nie im Frontend-Code
Token-Validierung	Jeder geschützte Endpunkt prüft den Token über Supabase
Eigentümer-Prüfung	<code>require_set_owner()</code> / <code>require_card_owner()</code> bei Schreibzugriffen
CORS	Konfiguriert in der Middleware; in Produktion auf die Frontend-URL einschränken
Sensible Daten	.env-Dateien liegen in .gitignore (Commit 6d2b4ad)

Sicherheits-Fix (06/2026): Der Passwort-Reset wurde von einem klickbaren Magic-Link auf einen manuell eingegebenen OTP-Code umgestellt. Grund: E-Mail-Sicherheitsscanner konsumierten den Einweg-Token des Links bereits vor dem Klick des Nutzers, wodurch der Reset clientseitig nie auf einer gültigen Session ankam.

9 Tests & Qualitätssicherung

Die Qualitätssicherung erfolgt zweistufig: strukturiertes manuelles Testen (nachverfolgt im Projektboard, Spalte „Review / Testing“) und automatisierte End-to-End-Tests mit Playwright, die alle manuell geprüften Szenarien reproduzierbar abdecken. Getestet wurde im Browser (Chrome und Firefox) sowie auf mobilen Viewports.

Durchgeführte manuelle Tests

Authentifizierung

Testfall	Ergebnis
Login mit gültigen Zugangsdaten	✓ Erfolgreich
Logout	✓ Session wird korrekt beendet
Gastansicht (ohne Login)	✓ Eingeschränkter Zugriff korrekt

Karteikarten & Lernsets

Testfall	Ergebnis
Karten erstellen	✓ Karte wird gespeichert und angezeigt
Karten laden	✓ Korrekt aus der Datenbank geladen
Karten bearbeiten	✓ Änderungen werden übernommen
Karten löschen	✓ Karte wird entfernt
Lernsets bearbeiten (Titel, Beschreibung)	✓ Erfolgreich
Lernsets löschen	✓ Set verschwindet aus der Liste
Private Lernsets	✓ Nur für Eigentümer sichtbar

Infrastruktur, Deployment & Gamification

Testfall	Ergebnis
Supabase-Verbindung	✓ Datenbankabfragen funktionieren korrekt
Deployment (Vercel/Production)	✓ Produktivumgebung erreichbar & funktionsfähig
Mobile Ansicht	✓ Layout passt sich kleinen Bildschirmen an
Streaks	✓ Lernserie wird korrekt gezählt und angezeigt
Guided Tour	✓ Startet bei Erstnutzung und ist überspringbar

Bekannte Bugs (identifiziert beim Testen)

Bug	Status
Sterne-Favoriten-Icon überlappt den „Karten“-Text in der Set-Karte	Offen
Startseite leitet in Firefox nicht auf das Dashboard weiter	Offen

Beispiel-Bugfix: Fork verlor Kartenreihenfolge (2026-06-01)

Problem: Beim Forken eines öffentlichen Sets wurden die Karteikarten ohne das position-Feld in die neue Kopie eingefügt. Dadurch verloren die Karten ihre Reihenfolge, da GET /sets/{set_id}/cards nach position sortiert.

Ursache: In backend/app/routers/sets.py fehlte "position": card.get("position", 0) beim Einfügen der kopierten Karten; außerdem wurden die Quellkarten nicht nach position abgefragt.

Fix: Quellkarten werden nun mit .order("position") abgefragt, und das position-Feld wird beim Insert in die Kopie übernommen.

Automatisierte E2E-Tests mit Playwright (ab 2026-06-01)

Die E2E-Tests befinden sich im Verzeichnis frontend/e2e/ und laufen automatisiert gegen die echte Supabase-Instanz. Voraussetzung: ein Test-Benutzer in Supabase sowie Zugangsdaten in frontend/.env.test:

```
TEST_USERNAME=testUser
TEST_PASSWORD=password
```

Ausführen:

```
cd studymate_v2/frontend
npm run test:e2e          # Alle Tests (Desktop + Mobile Chrome)
npm run test:e2e:ui       # Interaktiver UI-Modus
npm run test:e2e:report   # HTML-Bericht anzeigen
```

Testdatei	Abgedeckte Szenarien
auth.spec.js	Login, Registrierung, Logout, Gastansicht (9 Tests)
sets.spec.js	Supabase-Verbindung, Set CRUD, private Sets (7 Tests)
cards.spec.js	Karten laden/erstellen/bearbeiten/löschen (4 Tests)
ui.spec.js	Guided Tour, Streaks (5 Tests)
mobile.spec.js	Mobile-Ansicht auf Pixel-5-Viewport (3 Tests)
features.spec.js	Karten-Reihenfolge, Set kopieren, Gastfavoriten, KI-Quiz-Fehler (11 Tests)

Konfiguration (frontend/playwright.config.js): zwei Projekte — Desktop Chrome (alle Tests außer mobile.spec.js) und Mobile Chrome / Pixel 5 (nur mobile.spec.js). Der Dev-Server wird beim Start automatisch hochgefahren (reuseExistingServer: true).

- Tests verwenden Timestamp-basierte eindeutige Titel (PW_Set_{uid}) zur Testisolation.
- Erstellte Testdaten werden am Ende jedes Tests gelöscht (Cleanup in afterEach bzw. am Testende).
- .env.test ist in .gitignore eingetragen und wird nicht ins Repository eingchecked.

Empfohlene Backend-Tests (zukünftig, pytest)

```
pip install pytest pytest-asyncio httpx
pytest -q

def test_get_public_sets(client):
    response = client.get("/sets/public")
    assert response.status_code == 200
    assert isinstance(response.json(), list)

def test_fork_private_set_returns_403(client, auth_headers):
    response = client.post("/sets/private-set-id/fork", headers=auth_headers)
    assert response.status_code == 403

def test_fork_preserves_card_order(client, auth_headers,
public_set_with_ordered_cards):
    response = client.post(f"/sets/{public_set_with_ordered_cards}/fork",
headers=auth_headers)
    assert response.status_code == 200
    positions = [c["position"] for c in response.json()["flashcards"]]
    assert positions == sorted(positions)
```

10 Feature-Dokumentation (chronologisch)

Die folgende Aufstellung dokumentiert die Entwicklung commit-genau, gegliedert nach den fünf Projektphasen. Jede Phase entspricht grob einer Iteration und bündelt thematisch zusammengehörige Commits, sodass der Entwicklungsverlauf von den Grundfunktionen bis zum abschließenden Sicherheits-Fix nachvollziehbar bleibt.

Phase 1 — Grundfunktionen (2026-05-19)

Die erste Phase legte das Fundament der Anwendung: Authentifizierung über Supabase, die Datenbankanbindung sowie die Kernfunktionen rund um Kartensets, den Flip-Card-Lernmodus und eine erste KI-Quiz-Integration auf Basis von Google Gemini.

Datum	Commit	Feature
2026-05-19	8cea6d6	Supabase-Integration: Auth + Datenbankanbindung
2026-05-19	048da9b	Passwort-Reset-Flow mit E-Mail via Resend
2026-05-19	db4dcbb	KI-Quiz-Integration (Google Gemini)
2026-05-19	81b7561	Flip-Card-Animation (Performance-Optimierung)
2026-05-19	fa7c4df	Kartensets erstellen
2026-05-19	c2b967a	Karten bearbeiten und importieren
2026-05-19	7038f50	Verlinkung zur Anmeldeseite im Gastmodus

Phase 2 — Erweiterungen (2026-05-26)

In der zweiten Phase kamen zahlreiche Erweiterungen hinzu, die StudyMate von einer reinen Lern-App zu einer interaktiven Plattform ausbauten: Streak-Tracking zur Motivation, ein Light/Dark-Mode, die Fork-Funktion zum Kopieren öffentlicher Sets, eine geführte Tour für neue Nutzer sowie ein Profil-Overlay.

Datum	Commit	Feature
2026-05-26	28cf558	Streak-Tracking mit Datum-Parsing und Reset-Logik
2026-05-26	cb47bd8	Migration auf google-genai SDK; CORS-Fix
2026-05-26	dcc0679	Light/Dark-Mode
2026-05-26	9282ff3	Set-Name bearbeiten, Import-Funktion
2026-05-26	98d6cdf	Fork-Feature: Öffentliche Sets kopieren
2026-05-26	4a41540	Fork-UX: Dialog, bessere Ausrichtung
2026-05-26	409a1e3	Fork-Feature-Dokumentation
2026-05-26	322ab03	Geführte Tour mit localStorage und Restart-Button
2026-05-26	3d4d234	Profil-Overlay
2026-05-26	e46744c	onForkSet-Prop zur DetailView-Komponente

Phase 3 — Bugfixes & Verbesserungen (2026-05-26 – 2026-06-01)

Die dritte Phase diente der Stabilisierung. Im Fokus standen das Beheben gemeldeter Fehler, kleinere UX-Korrekturen sowie das Fixieren von Abhängigkeitsversionen, um einen reproduzierbaren Build sicherzustellen.

Datum	Commit	Beschreibung
2026-05-26	b332a07	Favoriten nur für eingeloggte Nutzer anzeigen
2026-05-26	f0e33f0	Benutzer-Feedback bei leeren Kartensets (Quiz/Lernen)
2026-05-26	0657acb	Edit-Icon in StudyMate-Komponente
2026-05-26	1a10046	Supabase-Version in requirements.txt fixiert
2026-05-26	184d550	Profil-Bug behoben
2026-06-01	cf603c0	Profilbild in NavBar/Sidebar; Learn/Quiz-Guard bei leeren Sets entfernt; handleAddCard-State-Dup gefixt

Phase 4 — Soziale Funktionen, KI-Anbieter-Wechsel, UI-Feinschliff (2026-06-01 – 2026-06-09)

Die vierte Phase brachte die größten funktionalen Sprünge: ein vollständiges Freunde-System mit öffentlichen Profilen, den Wechsel des KI-Anbieters von Google Gemini auf Groq, die Einführung des Playwright-E2E-Testframeworks sowie umfangreichen UI-Feinschliff.

Datum	Commit	Beschreibung
2026-06-01	5e42b21	KI-Anbieter von Gemini auf Groq (Llama 3.1) umgestellt (GEMINI_API_KEY → GROQ_API_KEY)
2026-06-01	70bb322	Aktivitäts-Tracking + Komponente WeeklyActivityChart
2026-06-01	a52e919	Eigenständige Sidebar-Komponente mit Navigation und Profilanzeige
2026-06-01	596b27b	Playwright-E2E-Testframework eingeführt (erste Testdateien)
2026-06-02	a15800c	Freunde-Feature (friendships, FriendsView, PublicProfileView); show_author-Dialog
2026-06-02	4ceec09	Neue Avatar-Komponente, Modal-Styling überarbeitet
06-02 – 06-08	mehrere	UI-Feinschliff: Sidebar-Einklapp-Button, Dark-Mode-Korrekturen, NavBar/Logo, Pfeil-Buttons
2026-06-08	fd41c1e	Globaler Exception-Handler im Backend; besseres KI-Quiz-Fehlerhandling
2026-06-08	06fea88	Hinweistext bei leerem Kartenset im Quiz-Modus
2026-06-08	5d401d4	Ladezustand Freundesliste; Tour um learn/quiz/friends/profile erweitert
2026-06-09	a50d7d1	Karten-Update-Logik überarbeitet; Spalte forked_from ergänzt
2026-06-09	18b6bcc / 4ddf635	WeeklyActivityChart: responsives Layout, Schriftskalierung, Farbschema

Phase 5 — Sicherheits-Fix Passwort-Reset (2026-06-23)

Die fünfte Phase bestand aus einem gezielten Sicherheits-Fix: Der Passwort-Reset wurde von einem klickbaren Magic-Link auf einen manuell einzugebenden OTP-Code umgestellt, nachdem E-Mail-Sicherheitsscanner die Einweg-Links vorzeitig entwertet hatten.

Datum	Commit	Beschreibung
2026-06-23	—	Passwort-Reset von Magic-Link auf OTP-Code umgestellt. ResetPasswordView und der PASSWORD_RECOVERY-Event-Handler entfernt; FRONTEND_URL ist kein benötigter Konfigurationswert mehr.

11 UI/UX — Design & Animationen

StudyMate verwendet ein konsistentes Design-System, das vollständig in StudyMate.jsx als injiziertes CSS definiert ist.

Design-System — Farbpalette

Farbe	Wert	Verwendung
Primärfarbe	#00d4aa	Buttons, Akzente, aktive Zustände
Hintergrund (Dark)	#080c18	App-Hintergrund
Text (primär)	#f1f5f9	Überschriften
Text (sekundär)	#64748b	Untertitel, Metadaten
Akzent violett	#8b5cf6	Quiz, KI-Features
Fehlerfarbe	#ef4444	Fehler, Löschen

Schriftarten: Sora (Interface), JetBrains Mono (Code, Zahlen).

Flip-Card-Animation

Die Lernkarten verwenden CSS-3D-Transformation. Wichtige CSS-Eigenschaften:

```
.sm-flip-scene { perspective: 1200px; }
.sm-flip-card { transform-style: preserve-3d;
                transition: transform .55s cubic-bezier(.4,0,.2,1); }
.sm-flip-card.flipped { transform: rotateY(180deg); }
.sm-flip-face { backface-visibility: hidden;
                -webkit-backface-visibility: hidden; }
.sm-flip-back { transform: rotateY(180deg) translateZ(1px); }
```

Bugfix-Hinweis (Firefox): `backface-visibility: hidden` muss auf beiden Seiten der Karte gesetzt sein, sonst kann in Firefox die Rückseite durchscheinen.

Responsive Design

- Desktop ($\geq 880\text{px}$): Sidebar dauerhaft sichtbar, Content verschoben (margin-left: 260px).
- Mobil ($< 880\text{px}$): Hamburger-Menü öffnet die Sidebar als Overlay.

12 CI/CD & Deployment

Frontend und Backend werden auf Vercel deployed. Das Frontend nutzt SPA-Routing (alle Pfade → index.html), das Backend läuft als Vercel Serverless Functions.

Deployment-Konfiguration

Frontend vercel.json (SPA-Routing):

```
{
  "rewrites": [{ "source": "/(.*)", "destination": "/" } ]
}
```

Backend vercel.json konfiguriert das Python-Backend als Vercel Serverless Functions.

Empfohlener CI/CD-Workflow

5. Push auf Feature-Branch
6. Linting (ESLint, ruff/flake8)
7. Unit-Tests (pytest, vitest)
8. Frontend-Build (npm run build)
9. Deploy auf Staging
10. PR-Review & Merge nach main
11. Deploy in Produktion

Branching-Strategie

Branch	Beschreibung
main	Produktionsstand
devi / Selina / Jenny / Michelle	Persönliche Arbeitsbranches

Pull Requests werden nach Code-Review in main gemerged.

13 Lokale Entwicklungsumgebung

Backend starten (Python 3.11+, pip)

```
cd studymate_v2/backend
python -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.local.example .env.local # Werte eintragen
uvicorn app.main:app --reload --port 8000
# Swagger-UI: http://localhost:8000/docs
```

Frontend starten (Node.js 18+, npm)

```
cd studymate_v2/frontend
npm install
# .env.local mit VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY, VITE_API_URL füllen
npm run dev                # http://localhost:5173
```

Umgebungsvariablen — Backend (.env)

Variable	Pflicht	Beschreibung
SUPABASE_URL	Ja	URL des Supabase-Projekts
SUPABASE_ANON_KEY	Ja	Supabase Anon/Public Key
SUPABASE_SERVICE_ROLE_KEY	Ja	Service Role Key (nur Backend!)
RESEND_API_KEY	Empfohlen	E-Mail-Versand (Passwort-Reset)
RESEND_FROM	Nein	Absenderadresse (Standard: onboarding@resend.dev)
GROQ_API_KEY	Optional	KI-Quiz (Llama 3.1)

Umgebungsvariablen — Frontend (.env.local)

Variable	Pflicht	Beschreibung
VITE_SUPABASE_URL	Ja	Supabase URL (öffentlich)
VITE_SUPABASE_ANON_KEY	Ja	Supabase Anon Key (öffentlich)
VITE_API_URL	Ja	Backend-URL (z. B. http://localhost:8000)

Sicherheitshinweis: SUPABASE_SERVICE_ROLE_KEY darf niemals im Frontend verwendet werden — er gewährt vollständigen DB-Zugriff ohne RLS-Einschränkungen.

14 Troubleshooting & FAQ

CORS-Fehler beim API-Aufruf

Ursache: Backend-URL falsch oder CORS nicht konfiguriert.

Lösung: VITE_API_URL im Frontend und CORS_ORIGINS im Backend prüfen. Backend nach .env-Änderung neu starten.

Supabase Token-Fehler (401)

Ursache: Abgelaufene Session oder falsche Keys.

Lösung: SUPABASE_URL und SUPABASE_ANON_KEY prüfen.

Passwort-Reset-Code wird abgelehnt („Code ungültig oder abgelaufen“)

Ursache: Code falsch eingegeben, älter als 1 Stunde, oder bereits einmal verwendet (jeder Code ist Einweg).

Lösung: Neuen Code über „Passwort vergessen?“ anfordern. Bei wiederholtem Auftreten: Supabase Dashboard → Authentication → Logs prüfen, ob der Code durch einen automatisierten Request (Mail-Scanner) bereits verbraucht wurde.

Flip-Card zeigt Vor- und Rückseite gleichzeitig (Firefox)

Lösung: CSS aus Kapitel 11 verwenden. Sicherstellen, dass backface-visibility: hidden auf beiden .sm-flip-face-Elementen gesetzt ist.

KI-Quiz schlägt fehl

Ursache: GROQ_API_KEY nicht konfiguriert.

Lösung: API-Key in Backend .env setzen. Die Antwort 503 „KI nicht konfiguriert“ bestätigt den fehlenden Key.

Passwort-Reset-E-Mail kommt nicht an

Ursache: RESEND_API_KEY fehlt oder Recovery-E-Mail im Profil nicht hinterlegt.

Lösung: Key in Backend .env setzen. Benutzer muss bei Registrierung eine Recovery-E-Mail angegeben haben.

Fork-Funktion gibt Fehler zurück

Ursache: Benutzer ist nicht eingeloggt oder versucht ein privates fremdes Set zu forken.

Lösung: Nur öffentliche (oder eigene) Sets können geforkt werden. Authentifizierungstoken muss gültig sein.

15 Versionshistorie (ChangeLog)

Version 0.3.2 (2026-06-23) — Aktuelle Version

Sicherheits-Fix: Passwort-Reset von klickbarem Magic-Link auf manuell eingegebenen 6-stelligen OTP-Code umgestellt (backend/app/routers/auth.py, frontend/src/components/AuthViews.jsx). Ursache war, dass E-Mail-Sicherheitsscanner den Einweg-Token des Links vor dem Klick des Nutzers verbrauchten. ResetPasswordView und der PASSWORD_RECOVERY-Event-Handler (onAuthStateChange) entfernt; Konfigurationsvariable FRONTEND_URL entfernt.

Aktualisierte Dokumentationsabschnitte: US-004, Abschnitt 6.2, 6.4, 7.2, 8, 10, 14, ChangeLog.

Version 0.3.1 (2026-06-09)

Schließt die Dokumentationslücke zwischen cf603c0 (2026-06-02) und 18b6bcc (2026-06-09) auf main.

- Freunde-Feature: Anfragen senden/annehmen/ablehnen/entfernen, Benutzersuche, öffentliche Profile (FriendsView, PublicProfileView, neue Tabelle friendships) — US-042, US-043.
- Autoren-Sichtbarkeits-Dialog (show_author) beim Veröffentlichen eines Sets.
- WeeklyActivityChart-Komponente im Dashboard inkl. responsivem Feinschliff und Farbschema.
- Eigenständige Sidebar-Komponente, neue Avatar-Komponente.
- Geführte Tour um learn, quiz, friends, profile erweitert.
- Spalte forked_from auf flashcard_sets; globaler Exception-Handler im Backend; Hinweistext bei leerem Kartenset im Quiz.

Breaking Change: KI-Anbieter von Google Gemini auf Groq (Llama 3.1 8B Instant) umgestellt — GEMINI_API_KEY existiert nicht mehr, ersetzt durch GROQ_API_KEY; Bibliothek google-genai durch openai (Groqs OpenAI-kompatibler Endpunkt) ersetzt.

Aktualisierte Dokumentationsabschnitte: US-012, US-042, US-043, Abschnitt 4, 5, 6.2, 6.4, 7.2, 7.3, 9, 13, 14, ChangeLog.

Version 0.3.0 (2026-06-02)

- Registrierung legt explizit einen profiles-Eintrag an (Benutzername-Eindeutigkeit vorab geprüft).
- Ladezustand beim App-Start (Spinner bis Auth-Status bekannt).
- Karten-Reihenfolge ändern (Pfeil-Buttons, position-Feld).
- Set kopieren (eigene öffentliche und private Sets); Priv-zu-Priv-Fork im Backend.

- KI-Quiz URL-Fallback auf `http://localhost:8000`; 503-Antworten mit benutzerfreundlicher Meldung.
- Gastfavoriten (`sm_favs_guest`) mit Migration beim Login; geführte Tour für Detail- & Erstellungsansicht.

Version 0.2.2 (2026-06-01)

- Bugfix: Fork kopiert Karten jetzt mit korrektem `position`-Feld — Reihenfolge bleibt erhalten.
- Manuelle Testdokumentation ergänzt.

Version 0.2.1 (2026-06-01)

- Profilbild als Avatar in NavBar und Sidebar-Fußzeile (Fallback: Anfangsbuchstabe).
- Learn-/Quiz-Buttons bei leeren Sets nicht mehr deaktiviert; leere-Set-Guard in LearnView entfernt.
- Bugfix: `handleAddCard` aktualisiert `currentSet`-State nicht mehr doppelt (verhinderte Kartenduplikate).

Version 0.2.0 (2026-06-01)

Geführte Tour (322ab03), Profil-Overlay & -bearbeitung (3d4d234, 184d550), Fork-Funktion (98d6cdf, 4a41540, e46744c), Set-Bearbeitung & JSON-Import (9282ff3), verbesserter Streak-Tracker (28cf558), Light/Dark-Mode (dcc0679), KI-Quiz-Migration auf google-genai (cb47bd8), Favoriten nur für eingeloggte Nutzer (b332a07), Feedback bei leeren Sets (f0e33f0).

Version 0.1.0 (2026-05-19) — Erstveröffentlichung

Supabase-Auth-Integration (8cea6d6), Passwort-Reset-Flow (048da9b), KI-Quiz-Grundstruktur (db4dcbb), Flip-Card-Lernmodus (81b7561), Kartensets erstellen (fa7c4df), Karten bearbeiten/importieren (c2b967a).

16 Offene Aufgaben & Ausblick

Priorität	Aufgabe
Hoch	Automatisierte Tests (Unit + E2E) weiter ausbauen
Hoch	Row Level Security (RLS) in Supabase konfigurieren
Mittel	Einstellungen-View vollständig implementieren
Mittel	sql_schema.sql an die Live-Datenbank synchronisieren (profiles-Erweiterungen, friendships, forked_from, show_author fehlen)
Mittel	OpenAPI-Export in docs/ ablegen
Niedrig	Profilbild-Speicherung zu Supabase Storage migrieren
Niedrig	Karteikarten-Bearbeitung über Backend-API statt direkt Supabase
Niedrig	Browser-Fallbacks für CSS-Features ergänzen

Anhang — Referenzen

A) Vollständiges SQL-Referenzschema

```
-- Lernsets
CREATE TABLE IF NOT EXISTS public.study_sets (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  owner_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  description TEXT,
  is_public BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Karteikarten
CREATE TABLE IF NOT EXISTS public.flashcards (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  set_id UUID NOT NULL REFERENCES public.study_sets(id) ON DELETE CASCADE,
  question TEXT NOT NULL,
  answer TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

B) Pydantic-Schemas (Übersicht)

app/schemas/sets.py

```
class StudySetCreate(BaseModel):
    title: str = Field(min_length=1, max_length=200)
    description: Optional[str] = Field(default=None, max_length=1000)
    is_public: bool = False

class StudySetUpdate(BaseModel):
    title: Optional[str] = Field(default=None, min_length=1, max_length=200)
    description: Optional[str] = Field(default=None, max_length=1000)
    is_public: Optional[bool] = None
```

app/schemas/quiz.py

```
class CardInput(BaseModel):
    q: str
    a: str

class QuizGenerateRequest(BaseModel):
    cards: list[CardInput]
    count: int = 5

class QuizQuestion(BaseModel):
    question: str
    options: list[str]
    correct: int
    explanation: str

class QuizGenerateResponse(BaseModel):
```

```
questions: list[QuizQuestion]
```

C) Nützliche curl-Befehle

```
# Health-Check
curl http://localhost:8000/health

# Öffentliche Sets abrufen
curl http://localhost:8000/sets/public

# Karten eines öffentlichen Sets
curl http://localhost:8000/sets/<set_id>/cards

# KI-Quiz generieren
curl -X POST http://localhost:8000/quiz/generate \
  -H "Content-Type: application/json" \
  -d '{"cards":[{"q":"Was ist SQL?", "a":"Structured Query Language"}], "count":1}'

# Öffentliches Set forken (mit Auth-Token)
curl -X POST http://localhost:8000/sets/<set_id>/fork \
  -H "Authorization: Bearer <token>"

# Passwort-Reset anfordern
curl -X POST http://localhost:8000/auth/forgot-password \
  -H "Content-Type: application/json" \
  -d '{"username":"max_mustermann"}'
```

D) JSON-Import-Format für Karteikarten

```
[
  {
    "question": "Was ist eine Firewall?",
    "answer": "Eine Sicherheitskomponente, die den Netzwerkverkehr filtert."
  },
  {
    "question": "Was bedeutet HTTPS?",
    "answer": "HyperText Transfer Protocol Secure – verschlüsselte HTTP-Kommunikation."
  }
]
```

Gesamtdokumentation des Projekts StudyMate · Stand 23.06.2026 · git main (PR #63).